

Essentials

Introduction:

Bash and shell scripting are powerful tools that allow you to automate tasks, streamline workflows, and enhance productivity, especially in Linux and Unix-based systems.

Whether you're a beginner or an experienced user, mastering Bash scripting can help you save time and become more efficient in your system administration and DevOps tasks.

Why Learn Bash Scripting?

- **Automate Repetitive Tasks:** With Bash scripts, you can easily automate tasks like backups, file manipulation, and system monitoring.
- **Improve System Administration Skills:** Bash scripting allows you to manage and troubleshoot systems with ease.
- **Enhance DevOps & Cloud Automation Workflows:** Automating tasks like deployment, scaling, and resource provisioning is critical in a DevOps or cloud environment.
- **Boost Productivity in Linux/Unix Environments:** Shell scripting is an essential skill for Linux and Unix users, enabling them to handle complex tasks efficiently.

Table of Contents:

1. **Getting Started with Bash Scripting**
2. **Basic Shell Commands**
3. **Variables and Data Types**
4. **Control Structures**
5. **Functions**
6. **Loops and Iteration**
7. **File Handling**
8. **Error Handling and Debugging**
9. **Best Practices for Writing Clean and Efficient Scripts**
10. **Practical Examples and Use Cases.**

1. Getting Started with Bash Scripting

To begin, let's look at how to create a basic Bash script.

Creating Your First Script

Create a New File: Open your terminal and create a new script file.

```
nano my_script.sh
```

1.

Add Shebang: At the beginning of your script, add the shebang (`#!/bin/bash`) to specify the script should run with Bash.

```
#!/bin/bash
```

2.

Make the Script Executable:

```
chmod +x my_script.sh
```

3.

Run the Script:

```
./my_script.sh
```

2. Basic Shell Commands

The shell is a command-line interface used to interact with an operating system. Below are some essential shell commands categorised based on their functionalities.

1. File and Directory Management

- `pwd` - Print the current working directory.
- `ls` - List files and directories.
 - `ls -l` (long listing format)
 - `ls -a` (show hidden files)
- `cd <directory>` - Change directory.
- `mkdir <directory>` - Create a new directory.
- `rmdir <directory>` - Remove an empty directory.
- `rm <file>` - Remove a file.
- `rm -r <directory>` - Remove a directory and its contents.
- `touch <file>` - Create an empty file.
- `cp <source> <destination>` - Copy a file or directory.
- `mv <source> <destination>` - Move or rename a file or directory.
- `find <directory> -name <filename>` - Search for a file.

2. File Viewing and Manipulation

- `cat <file>` - Display the content of a file.
- `tac <file>` - Display a file's content in reverse order.
- `less <file>` - View file content one page at a time.
- `head <file>` - Display the first 10 lines of a file.
- `tail <file>` - Display the last 10 lines of a file.
- `grep "pattern" <file>` - Search for a pattern in a file.
- `wc <file>` - Count lines, words, and characters in a file.
- `sort <file>` - Sort lines in a file.

3. File Permissions and Ownership

- `ls -l` - Show file permissions.
- `chmod <permissions> <file>` - Change file permissions.
 - Example: `chmod 755 script.sh`
- `chown <user>:<group> <file>` - Change file ownership.
- `umask` - View or set default permissions.

4. Process Management

- `ps` - Display active processes.
- `top` - Show real-time system processes.
- `kill <PID>` - Terminate a process.
- `pkill <process-name>` - Kill processes by name.
- `bg` - Resume a suspended process in the background.
- `fg` - Resume a background process in the foreground.
- `jobs` - List background processes.

5. Disk Usage and Monitoring

- `df -h` - Show disk space usage.
- `du -sh <directory>` - Show size of a directory.
- `free -h` - Display memory usage.
- `uptime` - Show system uptime.
- `who` - Show logged-in users.
- `history` - Show command history.
- `clear` - Clear the terminal screen.

6. Networking Commands

- `ping <host>` - Check network connectivity.
- `ifconfig` or `ip a` - Show network configuration.
- `netstat -tulnp` - Show active network connections.
- `curl <URL>` - Fetch content from a URL.
- `wget <URL>` - Download a file from a URL.

7. User Management

- `whoami` - Show current user.
- `id` - Display user and group IDs.
- `adduser <username>` - Add a new user.
- `passwd <username>` - Change user password.
- `groups <username>` - Show groups a user belongs to.

8. Archiving and Compression

- `tar -cvf archive.tar <files>` - Create a tar archive.
- `tar -xvf archive.tar` - Extract a tar archive.
- `gzip <file>` - Compress a file.
- `gunzip <file.gz>` - Decompress a file.
- `zip <archive.zip> <files>` - Create a zip file.
- `unzip <archive.zip>` - Extract a zip file.

9. Package Management

- **Debian-based (Ubuntu, Debian):**
 - `apt update` - Update package lists.
 - `apt install <package>` - Install a package.
 - `apt remove <package>` - Remove a package.

- **RedHat-based (RHEL, CentOS, Fedora):**
 - `yum update` - Update package lists.
 - `yum install <package>` - Install a package.
 - `yum remove <package>` - Remove a package.

10. Shell Scripting Basics

- `echo "Hello, World!"` - Print output to the terminal.
- `read var` - Read input from the user.
- `var=value` - Assign a variable.
- `export var=value` - Set an environment variable.
- `alias ll='ls -la'` - Create a shortcut command.
- `unalias ll` - Remove an alias.

3. Variables and Data Types in UNIX

In UNIX, variables are used to store and manipulate data within the shell environment.

Unlike some programming languages, the UNIX shell does not explicitly define data types but treats all variables as strings by default.

Below is an overview of variables and data handling in UNIX.

1. Defining and Using Variables

- Variables in UNIX are case-sensitive.
- Assign a value to a variable using `=` (no spaces around `=`).
- Access a variable using `$` before its name.

Example:

```
name="Alice"
```

```
echo "Hello, $name"
```

2. Types of Variables

a) Local Variables

- Defined within a script or session.
- Available only in the shell where they are set.

Example:

```
greeting="Welcome"
```

```
echo $greeting
```

b) Environment Variables

- Global variables available to all shell sessions.
- Use `export` to make a variable available system-wide.

Example:

```
export PATH="/usr/local/bin:$PATH"
```

```
echo $PATH
```

c) Shell Special Variables

- Predefined variables used by the shell.

Variable	Description
<code>0</code> <code>US\$</code>	Name of the script
<code>\$1,</code> <code>\$2, ...</code>	Positional parameters (arguments)
<code>\$#</code>	Number of arguments passed
<code>\$?</code>	Exit status of the last command
<code>\$\$</code>	Process ID of the current shell
<code>\$!</code>	Process ID of the last background job

Example:


```
echo "Script name: $0"
```

```
echo "First argument: $1"
```

```
echo "Process ID: $$"
```

3. Reading User Input

- The `read` command captures user input.

Example:

```
read -p "Enter your name: " user_name
```

```
echo "Hello, $user_name"
```

4. Default Values in Variables

- Using `: -` provides a default value if the variable is not set.

Example:

```
echo "User: ${USER:-guest}"
```

5. Arithmetic Operations

- Use `expr` or `$ ( ( ) )` for arithmetic calculations.

Example:

```
a=10
```

```
b=5
```

```
echo "Sum: $((a + b))"
```

```
echo "Product: $(expr $a \* $b)"
```

6. String Manipulation

- Extract substring, find length, replace values.

Example:

```
str="Hello UNIX"
```

```
echo "Length: ${#str}"
```

```
echo "Substring: ${str:0:5}"
```

```
echo "Replace: ${str/UNIX/Linux}"
```

7. Unset and Delete Variables

- `unset` removes a variable.

Example:

```
unset my_var
```

```
echo $my_var # No output
```

4. Control Structures in UNIX

Control structures in UNIX shell scripting help manage the flow of execution based on conditions and loops.

Below are the primary control structures used in UNIX shell scripts.

1. Conditional Statements

Conditional statements allow decision-making in scripts.

a) if Statement

The `if` statement evaluates conditions and executes commands accordingly.

Syntax:

```
if [ condition ]; then  
    # Commands to execute if condition is true  
fi
```

Example:

```
num=10  
  
if [ $num -gt 5 ]; then  
    echo "Number is greater than 5"  
fi
```

b) if-else Statement

Adds an alternative execution path.

Syntax:

```
if [ condition ]; then  
    # Commands if condition is true  
  
else  
    # Commands if condition is false  
fi
```

Example:

```
num=3

if [ $num -gt 5 ]; then

    echo "Number is greater than 5"

else

    echo "Number is 5 or less"

fi
```

c) if-elif-else Statement

Handles multiple conditions.

Syntax:

```
if [ condition1 ]; then

    # Commands if condition1 is true

elif [ condition2 ]; then

    # Commands if condition2 is true

else

    # Commands if none of the conditions are true

fi
```

Example:

```
num=7

if [ $num -gt 10 ]; then

    echo "Number is greater than 10"

elif [ $num -gt 5 ]; then

    echo "Number is between 6 and 10"
```

```
else
```

```
    echo "Number is 5 or less"
```

```
fi
```

2. Looping Constructs

Loops are used to execute commands repeatedly.

a) for Loop

Iterates over a range or list.

Syntax:

```
for var in list; do
```

```
    # Commands
```

```
done
```

Example:

```
for i in 1 2 3 4 5; do
```

```
    echo "Iteration $i"
```

```
done
```

b) while Loop

Executes commands while a condition is true.

Syntax:

```
while [ condition ]; do
```

```
    # Commands
```

```
done
```

Example:

```
count=1

while [ $count -le 5 ]; do

    echo "Count: $count"

    count=$((count + 1))

done
```

c) until Loop

Runs commands until a condition becomes true.

Syntax:

```
until [ condition ]; do

    # Commands

done
```

Example:

```
count=1

until [ $count -gt 5 ]; do

    echo "Count: $count"

    count=$((count + 1))

done
```

3. Case Statement

Similar to a switch statement in other languages.

Syntax:

```
case value in

    pattern1)
```

```
# Commands
```

```
::
```

```
pattern2)
```

```
# Commands
```

```
::
```

```
*)
```

```
# Default case
```

```
::
```

```
esac
```

Example:

```
read -p "Enter a number: " num
```

```
case $num in
```

```
1) echo "You entered one" ;;
```

```
2) echo "You entered two" ;;
```

```
*) echo "You entered something else" ;;
```

```
esac
```

5. Functions in UNIX Shell Scripting

Functions in UNIX shell scripting allow modularity and reusability of code, making scripts more structured and manageable.

They help in avoiding repetition and improve script efficiency.

1. Defining a Function

Functions are defined using the following syntax:

```
function_name() {  
    # Commands  
}
```

OR

```
function function_name {  
    # Commands  
}
```

2. Calling a Function

To execute a function, simply write its name:

```
function_name
```

Example:

```
say_hello() {  
    echo "Hello, User!"  
}
```

```
# Calling the function
```

```
say_hello
```

3. Passing Arguments to Functions

Functions can accept parameters, just like command-line arguments.

Example:

```
greet() {  
    echo "Hello, $1!"
```



```
}
```

```
greet "Alice"
```

```
greet "Bob"
```

Explanation:

- `$1`, `$2`, etc., represent the positional parameters passed to the function.

4. Returning Values from Functions

Functions can return exit status codes using the `return` command.

Example:

```
check_even() {
```

```
    if [ $((($1 % 2)) -eq 0 ]; then
```

```
        return 0 # Success
```

```
    else
```

```
        return 1 # Failure
```

```
    fi
```

```
}
```

```
check_even 4
```

```
if [ $? -eq 0 ]; then
```

```
    echo "Even number"
```

```
else
```

```
    echo "Odd number"
```

```
fi
```

Explanation:

- `$?` captures the return value of the function (0 for success, 1 for failure).

5. Using Local Variables in Functions

By default, variables inside functions are global. To avoid conflicts, use `local` to create function-scoped variables.

Example:

```
calculate_sum() {  
  
    local sum=$(( $1 + $2 ))  
  
    echo "Sum: $sum"  
  
}
```

```
calculate_sum 3 5
```

Explanation:

- `local sum` ensures `sum` is accessible only inside the function.

6. Functions with Command Substitution

Functions can return values using `echo` combined with command substitution.

Example:

```
get_date() {  
  
    echo $(date)  
  
}
```

```
today=$(get_date)  
  
echo "Today's date: $today"
```

Explanation:

- `$(get_date)` captures the function's output and stores it in `today`.

7. Recursive Functions

A function can call itself recursively, but caution is needed to avoid infinite loops.

Example:

```
factorial() {  
  
    if [ $1 -le 1 ]; then  
  
        echo 1  
  
    else  
  
        prev=$(factorial $(( $1 - 1 )))  
  
        echo $(( $1 * prev ))  
  
    fi  
  
}  
  
echo "Factorial of 5: $(factorial 5)"
```

Explanation:

- The function calls itself until it reaches `1`, then calculates factorial iteratively.

8. Function Libraries and Reusability

Functions can be stored in separate files and sourced into scripts to promote reusability.

Example:

Create a file `math_functions.sh`:

```
add() {  
    echo $(( $1 + $2 ))  
}
```

In another script:

```
source math_functions.sh  
  
result=$(add 3 7)  
  
echo "Sum: $result"
```

Explanation:

- `source` loads the functions from an external file.

9. Function Best Practices

- Use meaningful names for functions.
- Use `local` variables to prevent unintentional modifications.
- Use comments to describe function behavior.
- Store reusable functions in separate files and source them when needed.

Functions make UNIX shell scripts more readable, modular, and efficient. Using functions wisely enhances script maintainability and performance.

6. Loops and Iteration in UNIX Shell Scripting

Loops in UNIX shell scripting allow repeated execution of commands until a specific condition is met.

There are three primary types of loops: `for`, `while`, and `until`.

1. for Loop

The `for` loop iterates over a range of values or elements in a list.

Syntax:

```
for variable in list; do  
    # Commands to execute in each iteration  
done
```

Example 1: Iterating Over a List

```
for item in Apple Banana Cherry; do  
    echo "Fruit: $item"  
done
```

Output:

```
Fruit: Apple  
Fruit: Banana  
Fruit: Cherry
```

Example 2: Iterating Using a Range

```
for i in {1..5}; do  
    echo "Iteration $i"  
done
```

Output:

```
Iteration 1  
Iteration 2  
Iteration 3  
Iteration 4  
Iteration 5
```

2. while Loop

The `while` loop executes a set of commands as long as the condition is true.

Syntax:

```
while [ condition ]; do  
    # Commands  
done
```

Example 1: Counting from 1 to 5

```
count=1  
  
while [ $count -le 5 ]; do
```

```
    echo "Count: $count"

    count=$((count + 1))

done
```

Output:

Count: 1

Count: 2

Count: 3

Count: 4

Count: 5

Example 2: Reading a File Line by Line

```
while read line; do
```

```
    echo "Line: $line"
```

```
done < file.txt
```

3. until Loop

The `until` loop is similar to the `while` loop but runs until the condition becomes true.

Syntax:

```
until [ condition ]; do
```

```
    # Commands
```

```
done
```

Example: Counting from 1 to 5

```
count=1
```

```
until [ $count -gt 5 ]; do  
    echo "Count: $count"  
    count=$((count + 1))  
done
```

Output:

Count: 1

Count: 2

Count: 3

Count: 4

Count: 5

4. Loop Control Statements

Loop control statements modify the behaviour of loops.

a) break Statement

`break` exits the loop immediately.

Example:

```
for i in {1..5}; do  
    if [ $i -eq 3 ]; then  
        break  
    fi  
    echo "Iteration $i"  
done
```


Output:

Iteration 1

Iteration 2

b) continue Statement

`continue` skips the current iteration and moves to the next.

Example:

```
for i in {1..5}; do
    if [ $i -eq 3 ]; then
        continue
    fi
    echo "Iteration $i"
done
```

Output:

Iteration 1

Iteration 2

Iteration 4

Iteration 5

5. Nested Loops

Loops can be nested within one another to handle more complex scenarios.

Example: Printing a Pattern

```
for i in {1..3}; do
    for j in {1..3}; do
        echo -n "$i$j "
    done
done
echo ""
```

Output:

```
11 12 13
21 22 23
31 32 33
```

6. Using Loops with Command Substitution

Loops can process command output dynamically.

Example: Listing Files in a Directory

```
for file in $(ls); do
    echo "File: $file"
done
```

7. Best Practices for Using Loops

- Use `break` and `continue` judiciously to optimize performance.
- Prefer `while read` for file processing instead of `cat file | while read`.

- When iterating over large datasets, consider efficiency to prevent performance issues.
- Use `{1..N}` for range-based loops instead of `seq`, as it is more efficient.

Loops and iteration make shell scripting powerful by enabling automation of repetitive tasks efficiently.

7. File Handling in UNIX Shell Scripting

File handling in UNIX shell scripting allows users to create, read, write, modify, and delete files.

It is essential for automation and system administration tasks.

1. Creating Files

Using touch Command

```
touch myfile.txt
```

Creates an empty file named `myfile.txt`.

Using redirection operator

```
echo "Hello World" > myfile.txt
```

Creates `myfile.txt` and writes "Hello World" into it.

2. Reading Files

Using cat Command

```
cat myfile.txt
```

Displays the content of `myfile.txt`.

Using less Command

```
less myfile.txt
```

Opens `myfile.txt` in a paginated view.

Using head and tail Commands

```
head -5 myfile.txt # Displays the first 5 lines
```

```
tail -5 myfile.txt # Displays the last 5 lines
```

Using while Loop

```
while read line; do
```

```
    echo "$line"
```

```
done < myfile.txt
```

Reads file line by line.

3. Writing to Files

Using Redirection Operators

```
echo "New Line" > myfile.txt # Overwrites file
```

```
echo "Another Line" >> myfile.txt # Appends to file
```

Using Here Document

```
cat << EOF > myfile.txt
```

```
This is line 1
```

```
This is line 2
```

```
EOF
```

Writes multiple lines to `myfile.txt`.

4. Modifying Files

Using sed

```
sed -i 's/old/new/g' myfile.txt
```

Replaces "old" with "new" in `myfile.txt`.

Using awk

```
awk '{print $2}' myfile.txt
```

Prints the second column of each line in `myfile.txt`.

5. Deleting Files

```
rm myfile.txt # Deletes the file
```

6. Checking File Properties

Using test command

```
if [ -f myfile.txt ]; then
```

```
    echo "File exists"
```

```
fi
```

Checks if the file exists.

Using stat Command

```
stat myfile.txt
```

Displays detailed file information.

7. Changing File Permissions

Using chmod Command

`chmod 644 myfile.txt` # Sets read and write for owner, read-only for others

Using chown Command

`chown user:group myfile.txt` # Changes ownership

8. File Handling Best Practices

- Always check file existence before reading or writing.
- Use `>>` for appending rather than `>` to prevent accidental data loss.
- Use `sed` and `awk` for efficient text manipulation.
- Automate file backups before making major changes.

Efficient file handling is crucial for managing system operations, processing logs,

and automating administrative tasks in UNIX shell scripting.

8. Error Handling in Unix

Error handling involves managing errors and exceptions that may occur during the execution of commands or scripts.

Unix uses exit statuses and built-in commands for this purpose.

1. Exit Status Codes:

- Every command or script in Unix returns an exit status (also called exit code) after execution. A value of `0` means success, and any non-zero value indicates an error.
- To check the exit status of the last executed command, use the special variable `$?`.

Example:

```
ls /nonexistent_directory
```

```
echo $?
```

2. If the directory doesn't exist, the exit status will be non-zero (typically 2).

3. Using `set -e`:

- In a shell script, you can use `set -e` to make the script exit immediately if any command fails (i.e., returns a non-zero status).

Example:

```
set -e
```

```
command1 # If this fails, the script will stop here.
```

```
command2
```

4. Error Handling with `||`:

- The `||` operator allows you to define alternative commands in case the preceding command fails.

Example:

```
mkdir mydir || echo "Directory creation failed"
```

5. Error Handling with `if`:

- You can also use `if` conditions to check if a command succeeded or failed.

Example:

```
if ! mkdir mydir; then  
    echo "Failed to create directory"  
fi
```

Debugging in Unix

Debugging helps identify problems in commands or scripts. Below are methods for debugging Unix scripts.

1. Using `set -x`:

- This command prints each command before it is executed, along with its arguments, which helps in identifying where the problem lies.

Example:

```
set -x  
  
command1  
  
command2
```

2. Using `echo` Statements:

- For simple scripts, you can use `echo` to print out the values of variables and confirm the flow of execution.

Example:

```
echo "Starting command1"
```

```
command1
```

```
echo "Command1 finished"
```

3. Using `trap`:

- The `trap` command can be used to catch errors or signals and take specific actions (like printing debug info) when a signal or error occurs.

Example:

```
trap 'echo "Error on line $LINENO"; exit 1' ERR
```

4 . Using `ps`, `top`, or `htop` for Process Debugging:

- These commands help you see running processes and identify potential problems related to resource usage or deadlocks.

8 . Log Files:

- For complex systems or services, logs are often the key to debugging. You can check logs (like `/var/log/syslog` or `/var/log/messages`) to diagnose problems.

9 . Using `strace`:

- `strace` is a tool that helps you trace system calls made by a process. It's helpful when debugging issues related to system-level operations, such as file I/O or network communications.

Example:

`strace command_name`

10. Using `gdb` (GNU Debugger):

- `gdb` can be used for debugging compiled programs, allowing you to step through code, set breakpoints, and inspect variables.

Summary:

- Use **exit statuses** (`$?`) to handle errors.
- You can exit a script on error using `set -e` or check errors with `if`.
- **Debugging** can be done using `set -x`, `echo`, `trap`, and tools like `strace`, `gdb`, or checking logs.

9. Best Practices for Writing Clean and Efficient Scripts

Writing clean and efficient scripts in Unix is key to ensuring maintainability, readability, and performance.

Here are some best practices to consider when writing your scripts:

1. Use Meaningful Names for Variables and Functions

- Choose descriptive and meaningful names for variables, functions, and files.
- Avoid single-letter variables, except for loop counters.

Bad:

```
a="file1"
```

```
b="file2"
```

Good:

```
source_file="file1"
```

```
destination_file="file2"
```

2. Comment Your Code

- Use comments to explain what your code is doing, especially in complex or non-obvious sections.
- Ensure comments are concise but informative, and avoid stating the obvious.

Example:

```
# Checking if the file exists before copying
```

```
if [ -f "$source_file" ]; then
```

```
    cp "$source_file" "$destination_file"
```

```
fi
```

3. Follow Consistent Indentation

- Consistent indentation makes your scripts more readable.
- Use 2 or 4 spaces (avoid tabs), and be consistent throughout your script.

Example:

```
if [ -f "$file" ]; then
```

```
    echo "File found"
```

else

echo "File not found"

fi

4. Use Shell Built-ins for Efficiency

- Shell built-ins (e.g., `echo`, `test`, `[[`, `let`, `read`) are more efficient than external commands (like `ls`, `grep`, `awk`, etc.) because they don't spawn new processes.

Bad:

```
if [ "$(ls -A "$dir")" ]; then
```

```
    echo "Directory is not empty"
```

```
fi
```

Good:

```
if [ -n "$(find "$dir" -mindepth 1 -print -quit)" ]; then
```

```
    echo "Directory is not empty"
```

```
fi
```

5. Use `#!/bin/bash` (or another shell) at the Top of the Script

- Always specify the shell interpreter (e.g., `#!/bin/bash`) at the top of the script to ensure it runs with the correct shell.

Example:

```
#!/bin/bash
```

6. Check for Errors and Handle Them Gracefully

- Always check the success or failure of commands using exit status ($\$?$), and provide useful error messages.
- You can also use `set -e` to make the script exit if any command fails.

Example:

```
command || { echo "Command failed"; exit 1; }
```

7. Use Functions to Modularize Your Code

- Break your script into functions to promote code reuse, simplify debugging, and make the script more organized.

Example:

```
# Function to check if a directory exists
```

```
check_directory() {  
    if [ -d "$1" ]; then  
        echo "Directory $1 exists."  
    else  
        echo "Directory $1 does not exist."  
    fi  
}
```

8. Handle User Input Safely

- Always validate user input to prevent malicious code execution or unexpected behaviour.

Example:

```
read -p "Enter a filename: " filename

if [[ ! "$filename" =~ ^[a-zA-Z0-9._-]+$ ]]; then

    echo "Invalid filename"

    exit 1

fi
```

9. Use Loops for Repetitive Tasks

- Use loops (e.g., `for`, `while`, `until`) for tasks that need to be repeated.

Example:

```
for file in *.txt; do

    echo "Processing $file"

    # process the file

done
```

10. Avoid Using Absolute Paths When Possible

- Where possible, use relative paths or environment variables for file paths. This increases flexibility and portability.

Example:

```
# Instead of absolute paths
```

```
/home/user/scripts/myscript.sh
```

```
# Use relative paths or environment variables
```

```
$HOME/scripts/myscript.sh
```

11. Use `getopts` for Command-Line Arguments

- Use `getopts` for parsing command-line arguments in a structured and error-free manner.

Example:

```
while getopts "f:v" option; do
    case $option in
        f) file="$OPTARG";;
        v) verbose=true;;
        \?) echo "Usage: $0 [-f file] [-v]"; exit 1;;
    esac
done
```

12. Use `trap` for Cleanup

- Use the `trap` command to ensure cleanup tasks are performed when the script exits (e.g., deleting temporary files, closing file handles, etc.).

Example:

```
trap 'rm -f /tmp/tempfile' EXIT
```

13. Leverage `grep`, `awk`, `sed`, and Other Utilities Efficiently

- Use utilities like `grep`, `awk`, and `sed` wisely to process data. Avoid spawning new processes unnecessarily when a shell built-in can achieve the same result.

Bad:

```
cat file.txt | grep "pattern"
```

Good:

```
grep "pattern" file.txt
```

14. Minimize the Use of External Commands

- For simple tasks, avoid using external commands if you can achieve the result with shell built-ins (e.g., `test`, `[[`, `echo`, `printf`, etc.).

Bad:

```
count=$(wc -l < "$file")
```

Good:

```
count=$(( < "$file" wc -l ))
```

15. Use `printf` Instead of `echo` When Formatting Output

- `printf` is more consistent and portable than `echo` for formatting output (especially with special characters like newline, tabs, etc.).

Example:


```
printf "File: %s\n" "$filename"
```

16. Test Your Scripts in a Controlled Environment

- Always test your scripts in a non-production environment before running them on critical systems. This helps identify bugs and performance issues.

17. Avoid Hard-Coding Values

- Use variables for paths, filenames, and other values that may change. This makes the script more flexible and easier to maintain.

Example:

```
log_dir="/var/log/myapp"
```

```
log_file="$log_dir/app.log"
```

18. Handle Edge Cases

- Consider edge cases and error scenarios in your scripts. For example, what happens if a directory is empty, or a file doesn't exist?

19. Make Scripts Modular and Reusable

- Organize scripts into smaller, reusable components (e.g., functions, modules). This makes them easier to maintain and test.

20. Use `-n` for String Length Check

- Use `-n` when checking if a string is non-empty.

Example:

```
if [ -n "$variable" ]; then
```

```
echo "String is not empty"
```

```
fi
```

10. Practical Examples and Use Cases

Here are some practical examples and use cases for a **DevOps Cloud Engineer** working with Unix/Linux environments, which can help in various aspects of cloud infrastructure management, automation, CI/CD pipelines, and system administration.

1. Automating EC2 Instance Setup with Terraform and Bash Scripts

- **Use Case:** Automating the provisioning and configuration of EC2 instances in AWS with Terraform and bash scripts.
- **Example:**
 - A typical `terraform` configuration to spin up an EC2 instance, followed by a bash script to configure it.

```
# Terraform configuration to create EC2 instance
```

```
resource "aws_instance" "my_instance" {  
  
    ami          = "ami-0c55b159cbfafa1f0"  
  
    instance_type = "t2.micro"  
  
}
```

```
# After creating the EC2 instance, SSH into it using a bash script
```

```
instance_ip=$(terraform output my_instance_public_ip)
```

```
ssh -i ~/.ssh/my-key.pem ubuntu@$instance_ip << 'EOF'
```

```
sudo apt update && sudo apt install -y docker.io
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
EOF
```

2. Automating Deployment Using CI/CD (Jenkins with Docker)

- **Use Case:** Automating application deployment using Jenkins pipelines integrated with Docker.
- **Example:**
 - Jenkinsfile to build and deploy a Dockerized application:

```
pipeline {  
    agent any  
    environment {  
        DOCKER_IMAGE = "myapp:v1"  
    }  
    stages {  
        stage('Build') {  
            steps {  
                script {  
                    docker.build(DOCKER_IMAGE)  
                }  
            }  
        }  
    }  
}
```

```
}  
  
}  
  
stage('Push') {  
    steps {  
        script {  
            docker.withRegistry("", 'dockerhub-creds') {  
                docker.image(DOCKER_IMAGE).push()  
            }  
        }  
    }  
}  
  
stage('Deploy') {  
    steps {  
        script {  
            sh 'ssh -i ~/.ssh/my-key.pem user@server "docker pull myapp:v1 &&  
docker run -d myapp:v1"'  
        }  
    }  
}  
}
```

3. Automating Infrastructure with Ansible for Cloud Servers

- **Use Case:** Using Ansible to automate the configuration of cloud servers like EC2 instances or Azure VMs.
- **Example:**
 - Ansible playbook to install Docker and deploy an application:

- name: Install Docker on cloud servers

hosts: all

become: yes

tasks:

- name: Install Docker

apt:

name: docker.io

state: present

- name: Start Docker service

service:

name: docker

state: started

enabled: yes

- name: Deploy app container

docker_container:

name: myapp

image: myapp:v1

state: started

4. Log Monitoring and Alerts with `syslog` and `logrotate`

- **Use Case:** Setting up centralized logging for monitoring and alerting in Unix systems.
- **Example:**
 - Configure `logrotate` to manage log files and prevent them from growing too large:

```
/var/log/myapp/*.log {  
  
    daily  
  
    rotate 7  
  
    compress  
  
    missingok  
  
    notifempty  
  
    create 0644 root root  
}
```

- Use `syslog` to forward logs to a centralized logging system (like ELK Stack or CloudWatch):

```
# /etc/rsyslog.conf  
  
*. * @logs.example.com:514
```

5. Automating Cloud Infrastructure with AWS CLI and Bash

- **Use Case:** Automating infrastructure provisioning in AWS using AWS CLI combined with Bash scripts.

- **Example:**
 - Bash script to create a security group and launch an EC2 instance:

```
# Create a security group
```

```
security_group_id=$(aws ec2 create-security-group --group-name my-security-group --description "My security group" --query 'GroupId' --output text)
```

```
# Open port 22 for SSH access
```

```
aws ec2 authorize-security-group-ingress --group-id $security_group_id --protocol tcp --port 22 --cidr 0.0.0.0/0
```

```
# Launch an EC2 instance using the security group
```

```
aws ec2 run-instances --image-id ami-0c55b159cbfafe1f0 --instance-type t2.micro --security-group-ids $security_group_id --key-name my-key
```

6. Continuous Monitoring with Prometheus and Grafana on Kubernetes

- **Use Case:** Setting up monitoring for cloud-native applications using Prometheus and Grafana within a Kubernetes cluster.
- **Example:**
 - Use Helm to install Prometheus and Grafana on Kubernetes:

```
helm install prometheus prometheus-community/kube-prometheus-stack
```

```
helm install grafana grafana/grafana
```

- Expose the Grafana dashboard to view metrics and logs.
- Use `kubectl` to create a service to expose Grafana and Prometheus:

```
kubectl expose service prometheus-operated --type=LoadBalancer --name=prometheus
```

```
kubectl expose service grafana --type=LoadBalancer --name=grafana
```

7. Scaling Applications Using Kubernetes and Helm

- **Use Case:** Automating scaling of applications in a Kubernetes cluster using Helm charts.
- **Example:**
 - Helm chart to scale a microservices-based app in Kubernetes:

```
# Install the application using Helm
```

```
helm install myapp ./myapp-chart
```

```
# Scale the deployment
```

```
kubectl scale deployment myapp-deployment --replicas=3
```

8. Automating Backup and Restore of Databases Using Cron Jobs

- **Use Case:** Automating database backups and restores on a scheduled basis.
- **Example:**
 - Backup MySQL database with a cron job:

```
# Add to crontab (run every day at 2 AM)
```

```
0 2 * * * /usr/bin/mysqldump -u root -p'password' my_database >  
/backup/my_database_$(date +%F).sql
```

- - To restore:

```
mysql -u root -p'password' my_database < /backup/my_database_2025-  
02-25.sql
```


9. Managing and Rolling Back Application Deployments with Helm and Kubernetes

- **Use Case:** Rolling back an application deployment in Kubernetes using Helm.

Example:

Check the history of the Helm release

```
helm history myapp
```

Rollback to the previous version

```
helm rollback myapp 1
```

10. Security and Compliance Audits with OpenSCAP

- **Use Case:** Running security and compliance audits on Linux servers using OpenSCAP.
- **Example:**
 - Run an audit for compliance using OpenSCAP:

```
oscap oval eval --results /tmp/scan_results.html --report  
/tmp/scan_report.html /usr/share/openscap/scap-yaml/ssg-centos7-ds.xml
```

These examples are designed to help a DevOps Cloud Engineer automate and optimize cloud infrastructure, manage deployments, and monitor the system efficiently.